

```

1  /* =====
2      kreisgrafik.js – Die Kreisdiagramme der Orte
3
4      Je Ort ein Kreis: Grösse zählt die Erwähnungen, Form zeigt die Valenz.
5      Unten schraffierte Gesamtkreise je Kategorie, darüber die Valenzflächen
6      (Halbkreis neg/pos, Vollkreis neutral), aussen herum ein F-Wert-Punkt je
7      Annotation. Winkel-Konvention und Abhängigkeiten: docs/architektur.md.
8  ===== */
9
10 // --- Modulkapselung -----
11 // 86 von 99 Namen intern, 13 exportiert. Konvention: docs/architektur.md.
12 // ACHTUNG Ladezeit: hexZuRgb() und baueDemoAnnotationen() laufen schon in der
13 // IIFE – diese Datei muss nach datenbereinigung.js stehen, sonst ReferenceError.
14 // p5s Konstanten (PI, HALF_PI) gibt es hier noch nicht, die setzt p5 erst beim
15 // Start des Sketches. Auf Modulebene deshalb nur Math.PI.
16 (function () {
17
18     // ACHTUNG p5s text()/arc()/ellipse() bleiben bei laufender Animation
19     // manchmal unsichtbar – deshalb wird hier direkt in drawingContext
20     // gezeichnet. Das lässt p5s Farb-Zwischenspeicher veralten; nur pop()
21     // gleicht ihn wieder ab, ctx.restore() nicht. Jede Zeichenfunktion klammert.
22
23     // Zeilenabstand der Schraffur in den Gesamtkreisen.
24     const HATCH_SPACING = 3;
25
26     // Zeilenmass der Kreis-Labels; zeichneKreisLabels rutscht damit aus, die
27     // Erklärungs-Ebene rechnet damit ihren Platzbedarf aus.
28     const LABEL_HOEHE = 14;
29     const LABEL_ABSTAND = 4;
30     // Schriftgrad aller Beschriftungen im Canvas: Ortsnamen, Legendentitel,
31     // Blockzeilen, Valenz- und Wahrnehmungslabels.
32     // ACHTUNG .annotation-tag in style.css führt denselben Wert als font-size –
33     // die Kategorienzeile der Annotationsbox ist DOM, nicht Canvas. Wird der Wert
34     // hier geändert, muss er dort mit, sonst laufen die beiden auseinander.
35     const LABEL_GROESSE = 13;
36
37     // Die echten Ortskreise dieses Frames – Grundlage der Erklärungs-Ebene, die
38     // sich an den grössten davon hängt. frameCount setzt die Liste selbst
39     // zurück, damit kein Aufrufer daran denken muss.
40
41     // ACHTUNG die Liste wird nur beim ersten merkeKreis() eines Frames geleert.
42     // Wird in einem Frame gar keiner gezeichnet, steht noch der Stand des
43     // letzten drin – deshalb liest kreiseDiesesFrames() den Zähler mit.
44     let gezeichneteKreise = [];
45     let kreisRegisterFrame = -1;
46
47     function kreiseDiesesFrames() {
48         return kreisRegisterFrame === frameCount ? gezeichneteKreise : [];
49     }
50
51     function merkeKreis(cx, cy, bandCounts, radius, fwertPunkte, skala = 1, maxRadius = 100) {
52         if (kreisRegisterFrame !== frameCount) { gezeichneteKreise = []; kreisRegisterFrame = frameCount; }
53         if (radius > 0) gezeichneteKreise.push({ cx, cy, bandCounts, radius, fwertPunkte, skala, maxRadius });
54     }
55
56     // Ein background()-Aufruf übermalt den bisherigen Frame – was vorher
57     // gemerkt wurde, ist nicht mehr zu sehen.
58     function vergissGezeichneteKreise() {
59         gezeichneteKreise = [];
60         kreisRegisterFrame = frameCount;
61     }
62
63     // ACHTUNG gehört zur stillgelegten neutralen Vollfläche in
64     // zeichneKreisFuerRun und hat solange keinen Leser. Bleibt stehen, weil der

```

```

65 // Aufruf nur auskommentiert ist – wie zeichneWindrose in kartendekor.js.
66 const NEUTRAL_DAEMPfung = 0.35;
67
68 // -----
69 // Kreise
70 // -----
71
72 function drawHatchedCircle(cx, cy, r, color, alphaSkala = 1) {
73   if (r <= 0) return;
74   push(); // schreibt strokeStyle direkt, siehe ACHTUNG oben
75   const ctx = drawingContext;
76   ctx.beginPath();
77   ctx.arc(cx, cy, r, 0, Math.PI * 2);
78   ctx.clip();
79   ctx.strokeStyle = color;
80   ctx.globalAlpha = 0.55 * alphaSkala;
81   ctx.lineWidth = 1.8;
82   for (let ly = cy - r; ly <= cy + r; ly += HATCH_SPACING) {
83     ctx.beginPath();
84     ctx.moveTo(cx - r, ly);
85     ctx.lineTo(cx + r, ly);
86     ctx.stroke();
87   }
88   pop();
89 }
90
91 // -----
92 // Route
93 // -----
94
95 function leereBandCounts() {
96   return {
97     gold_dunkel: { neg: 0, pos: 0, neutral: 0, unrated: 0 },
98     gold_mittel: { neg: 0, pos: 0, neutral: 0, unrated: 0 },
99     gold_hell: { neg: 0, pos: 0, neutral: 0, unrated: 0 },
100   };
101 }
102
103 function zeichneKreiseOrtRuns(punktIndex, annIndex, activeBbox, offsetX = mapOffsetX, offsetY =
mapOffsetY, daten = stationenData) {
104   let runs = daten.ortRuns || [];
105   let labelKandidaten = [];
106
107   runs.forEach(r => {
108     // Wer einen Kreis bekommt, entscheidet datenbereinigung.js – vier
109     // Ausschlussgründe, alle Aussagen über die Daten.
110     if (!ortRunSichtbar(r, punktIndex, annIndex, daten)) return;
111     let pos = lonLatToScreen(r.lon, r.lat, activeBbox, offsetX, offsetY);
112     let filter = wohnungFilterFuerOrt(r.ort);
113     // EIN Durchlauf für beides: bandCounts und dieselbe Trefferliste für
114     // die F-Wert-Punkte unten.
115     let treffer = sammleAnnotationenNachOrtBasis(filter, annIndex, daten);
116     let bandCounts = zaehleBandCounts(treffer);
117     // winkel PI wie in der Graph-Ansicht: positiv oben, negativ unten.
118     let radius = groessterKreisRadius(bandCounts);
119     zeichneKreiseFuerRun(pos.x, pos.y, bandCounts, 1, PI);
120     let fwertAnnotationen = treffer.filter(a => a.hasFwert);
121     merkeKreis(pos.x, pos.y, bandCounts, radius,
122       zeichneFwertPunkte(pos.x, pos.y, radius, fwertAnnotationen, 1));
123     if (radius > 0) {
124       // Erst sammeln, Kollisionen nach der Schleife (zeichneKreisLabels).
125       // Der Routen-Startpunkt blendet erst mit dem Kapitel-1-Ausschnitt ein.
126       let istRoutenStart = daten === stationenData && r.ort === WOHNUNG_SAMMELPUNKT_ANKER;
127       labelKandidaten.push({
128         ankerX: pos.x, ankerY: pos.y,
129         x: pos.x, y: pos.y + 15,

```

```

130     text: r.ort.toUpperCase(), // .annotation-tag ist text-transform: uppercase
131     farbe: null,
132     alpha: istRoutenStart ? kapitel1ZoomAmount : 1,
133   });
134 }
135 });
136
137 zeichneKreisLabels(labelKandidaten);
138 }
139
140 // Platziert die Labels von oben nach unten und rutscht bei Überlappung
141 // nach unten. Ab spürbarem Versatz zieht eine gestrichelte Linie zum Kreis.
142 function zeichneKreisLabels(kandidaten) {
143   // Alpha 0 fällt ganz raus: kein Platz im Layout, keine Hilfslinie.
144   kandidaten = kandidaten.filter(k => (k.alpha === undefined ? 1 : k.alpha) > 0.002);
145   if (kandidaten.length === 0) return;
146
147   push(); // sechs Zeichenzustände plus direkte fillStyle-Schreibzugriffe
148   noStroke();
149   fill(33, 43, 46, 255); // #212B2E, wie die Kapitelnummern
150   beschriftungsSchrift(LABEL_GROESSE);
151   textAlign(LEFT, CENTER);
152
153   let platziert = [];
154
155   kandidaten
156     .map(k => ({ ...k, w: textWidth(k.text) }))
157     // links: der Text endet am Ankerplatz, statt dort zu beginnen.
158     .map(k => k.links ? { ...k, x: k.x - k.w } : k)
159     .sort((a, b) => a.y - b.y)
160     .forEach(k => {
161       let y = k.y;
162       let ueberlappt = true;
163       while (ueberlappt) {
164         ueberlappt = platziert.some(p =>
165           y < p.y + LABEL_HOEHE + LABEL_ABSTAND && y + LABEL_HOEHE + LABEL_ABSTAND > p.y &&
166           k.x < p.x + p.w && k.x + k.w > p.x
167         );
168         if (ueberlappt) y += LABEL_HOEHE + LABEL_ABSTAND;
169       }
170       platziert.push({ x: k.x, y, w: k.w });
171
172       let alpha = k.alpha === undefined ? 1 : k.alpha;
173       // hilfslinie erzwingt die Linie auch ohne Ausweichen (Demo-Grafik).
174       if (k.hilfslinie || Math.abs(y - k.y) > 1) {
175         stroke(0, 255 * alpha);
176         strokeWeight(0.8);
177         drawingContext.setLineDash([2, 3]);
178         line(k.ankerX, k.ankerY, k.links ? k.x + k.w + 4 : k.x - 4, y);
179         drawingContext.setLineDash([]);
180         noStroke();
181       }
182       // Direkt statt fill(): die Farbe wechselt je Label (alpha).
183       drawingContext.fillStyle = k.farbe
184         ? k.farbe
185         : `rgba(33, 43, 46, ${alpha})`;
186       // Siehe ACHTUNG oben.
187       drawingContext.fillText(k.text, k.x, y);
188     });
189
190   pop();
191 }
192
193 // winkelMitte = Bildschirmwinkel der Wölbungsmitte (0 = rechts, im
194 // Uhrzeigersinn).
195 // ACHTUNG deckend gezeichnet, ohne Multiply und ohne festen Alpha-Abschlag.

```

```

196 // Beides zusammen verschob die Farbe gegenüber KREIS_KATEGORIEN – die Legende
197 // zeigte daneben einen anderen Ton als der Kreis. alphaSkala bleibt: es blendet
198 // die ganze Grafik ein und aus.
199 function zeichneHalbkreis(cx, cy, r, winkelMitte, farbeRgb, alphaSkala = 1) {
200   if (r <= 0) return;
201   push();
202   let ctx = drawingContext;
203   ctx.fillStyle = `rgba(${farbeRgb[0]}, ${farbeRgb[1]}, ${farbeRgb[2]}, ${alphaSkala})`;
204   ctx.beginPath();
205   ctx.moveTo(cx, cy);
206   ctx.arc(cx, cy, r, winkelMitte - HALF_PI, winkelMitte + HALF_PI);
207   ctx.closePath();
208   ctx.fill();
209   pop();
210 }
211
212 // Neutral hat keine Seite, deshalb ganze Fläche statt Halbkreis.
213 // ACHTUNG derzeit ohne Aufrufer, siehe die stillgelegte neutrale Vollfläche in
214 // zeichneKreiseFuerRun.
215 function zeichneVollkreis(cx, cy, r, farbeRgb, alphaSkala = 1) {
216   if (r <= 0) return;
217   push();
218   let ctx = drawingContext;
219   ctx.fillStyle = `rgba(${farbeRgb[0]}, ${farbeRgb[1]}, ${farbeRgb[2]}, ${alphaSkala})`;
220   ctx.beginPath();
221   ctx.arc(cx, cy, r, 0, TWO_PI);
222   ctx.fill();
223   pop();
224 }
225
226 // winkel: feste Basis der Valenz-Teilung (PI für Karte/Graph, Default
227 // -HALF_PI für die Ortsveränderung). radiusSkala/maxRadius nur dort genutzt.
228
229 // ACHTUNG kein Rückgabewert. Radius holt groessterKreisRadius() – dessen
230 // letzte zwei Parameter stehen UMGEKEHRT zu denen hier.
231 function zeichneKreiseFuerRun(cx, cy, bandCounts, alphaSkala = 1, winkel = -HALF_PI, radiusSkala =
1, maxRadius = 100) {
232   // Schraffur immer unten, Valenzflächen darüber; innerhalb jeder Ebene
233   // grösste zuunterst. Sonst deckt eine Schraffur fremde Flächen zu.
234   push(); // schreibt unten direkt in fillStyle (Mittelpunkt)
235   let hatchFormen = [];
236   let flaechenFormen = [];
237   // Dieselbe Funktion, die auch die Aufrufer vor dem Zeichnen benutzen.
238   let aussenRadius = groessterKreisRadius(bandCounts, maxRadius, radiusSkala);
239
240   KREIS_KATEGORIEN.forEach(k => {
241     let bc = bandCounts[k.key] || {};
242     let n = (bc.neg || 0) + (bc.pos || 0) + (bc.neutral || 0) + (bc.unrated || 0);
243     let hatchR = kreisRadius(n, maxRadius) * radiusSkala;
244     if (hatchR > 0) {
245       let hex = rgbZuHex(k.farbe);
246       hatchFormen.push({ r: hatchR, zeichne: () => drawHatchedCircle(cx, cy, hatchR, hex,
alphaSkala) });
247     }
248
249     // Alle drei Ansichten übergeben winkel PI: positiv oben, negativ unten.
250     let negR = kreisRadius(bc.neg || 0, maxRadius) * radiusSkala;
251     let posR = kreisRadius(bc.pos || 0, maxRadius) * radiusSkala;
252     if (negR > 0) flaechenFormen.push({ r: negR, zeichne: () => zeichneHalbkreis(cx, cy, negR,
winkel - HALF_PI, k.farbe, alphaSkala) });
253     if (posR > 0) flaechenFormen.push({ r: posR, zeichne: () => zeichneHalbkreis(cx, cy, posR,
winkel + HALF_PI, k.farbe, alphaSkala) });
254     // Die neutrale Vollfläche ist stillgelegt: sie legte sich als geschlossene
255     // Scheibe über beide Hälften und machte die Mitte unlesbar. Neutrales zählt
256     // weiterhin in die Schraffur und damit in den Aussenradius.
257     // let neutralR = kreisRadius(bc.neutral || 0, maxRadius) * radiusSkala;

```

```

258 // if (neutralR > 0) flaechenFormen.push({ r: neutralR, zeichne: () => zeichneVollkreis(cx, cy,
    neutralR, k.farbe, alphaSkala * NEUTRAL_DAEMPfung) });
259 });
260
261 hatchFormen.sort((a, b) => b.r - a.r).forEach(f => f.zeichne());
262 flaechenFormen.sort((a, b) => b.r - a.r).forEach(f => f.zeichne());
263
264 if (ausserRadius > 0) {
265     // Mittelpunkt. Canvas-Pfad, siehe ACHTUNG oben.
266     drawingContext.fillStyle = `rgba(0, 0, 0, ${alphaSkala})`;
267     drawingContext.beginPath();
268     drawingContext.arc(cx, cy, 4, 0, TWO_PI);
269     drawingContext.fill();
270 }
271 pop();
272 }
273
274 // Ringabstände der F-Wert-Punkte; die Durchmesser selbst stehen als
275 // FWERT_PUNKT_DURCHMESSER in datenbereinigung.js, weil fotomarker.js sie
276 // ebenfalls liest.
277 const FWERT_PUNKT_RAND_ABSTAND = 6; // Luft zwischen Kreisrand und erstem Punkte-Ring
278 const FWERT_PUNKT_RING_ABSTAND = 8; // Abstand zwischen zwei Punkte-Ringen, falls ein Abschnitt
    nicht in einen Ring passt
279 const FWERT_PUNKT_LUECKE = 2; // Mindestabstand zwischen benachbarten Punkten
280
281 // Mitte des 120°-Drittels je Valenzgruppe: neg unten, pos oben, neutral rechts.
282 // Einzige Quelle dieser Konvention – zeichneFwertPunkte() setzt die Punkte
283 // danach, demoBeschriftungen() zeigt mit den Hilfslinien darauf.
284 // Math.PI statt HALF_PI: siehe ACHTUNG zur Ladezeit oben.
285 // Jede Valenzgruppe bekommt einen Bogenabschnitt von 100°, dazwischen 20°
286 // Luft – 3 × 100° + 3 × 20° ergeben die vollen 360°. Neutral zeigt nach
287 // rechts, positiv nach links oben, negativ nach links unten (Bildschirmwinkel:
288 // 0 = rechts, im Uhrzeigersinn). Siehe docs/topografie-der-gefuehle-grafik.pdf.
289 // Math.PI statt radians(): die Konstanten entstehen beim Laden des Moduls,
290 // p5-Globals stehen da noch nicht sicher bereit.
291 const FWERT_GRUPPEN_SPANNE = Math.PI * 100 / 180;
292 const FWERT_GRUPPEN_LUECKE = Math.PI * 20 / 180;
293 const FWERT_GRUPPEN_VERSATZ = FWERT_GRUPPEN_SPANNE + FWERT_GRUPPEN_LUECKE;
294 const FWERT_GRUPPEN_WINKEL = { neg: FWERT_GRUPPEN_VERSATZ, pos: -FWERT_GRUPPEN_VERSATZ, neutral: 0
    };
295
296 // Ein Punkt je Annotation mit F-Wert, Grösse nach Typ, Lage im 120°-Drittel
297 // der eigenen Valenz. Bei Andrang wachsen weitere Ringe nach aussen.
298 // Rückgabe: die gesetzten Punkte, an die sich die Erklärungs-Ebene hängt.
299 function zeichneFwertPunkte(cx, cy, radius, fwertAnnotationen, alphaSkala = 1) {
300     let gesetzt = [];
301     if (!fwertAnnotationen.length || radius <= 0) return gesetzt;
302
303     push(); // noStroke() plus direkte fillStyle-Schreibzugriffe
304     // Reihenfolge [neg, pos, neutral] – gruppen[0..2] unten wird so indiziert.
305     let mitten = [FWERT_GRUPPEN_WINKEL.neg, FWERT_GRUPPEN_WINKEL.pos, FWERT_GRUPPEN_WINKEL.neutral];
306     let gruppen = mitten.map(mitte => ({ mitte, formen: [] }));
307     fwertAnnotationen.forEach(a => {
308         let gruppe = a.valenz === -1 ? gruppen[0] : a.valenz === 1 ? gruppen[1] : gruppen[2];
309         let groesse = FWERT_PUNKTGROESSE[a.fWertType] || 1;
310         gruppe.formen.push({
311             d: FWERT_PUNKT_DURCHMESSER[groesse],
312             rgb: FWERT_COLOR_RGB,
313             typ: a.fWertType,
314         });
315     });
316
317     noStroke();
318     gruppen.forEach(({ mitte, formen }) => {
319         if (!formen.length) return;
320         let ringRadius = radius + FWERT_PUNKT_RAND_ABSTAND;

```

```

321   let rest = formen;
322   while (rest.length) {
323     // Platzbudget eines Rings: der ganze Bogenabschnitt. Die 20° Luft zu
324     // den Nachbarn hält die Gruppen auseinander, auch wenn er voll wird.
325     let bogenlaenge = ringRadius * FWERT_GRUPPEN_SPANNE;
326     let platz = 0;
327     let anzahlImRing = 0;
328     for (let f of rest) {
329       let breite = f.d + FWERT_PUNKT_LUECKE;
330       if (anzahlImRing > 0 && platz + breite > bogenlaenge) break;
331       platz += breite;
332       anzahlImRing++;
333     }
334     let ringFormen = rest.slice(0, anzahlImRing);
335     rest = rest.slice(anzahlImRing);
336
337     // Die Punkte wachsen aus der Mitte ihres Abschnitts heraus: die Reihe ist
338     // nur so breit, wie sie sein muss, und sitzt mittig auf `mitte`. Früher
339     // spannte sie sich über die vollen 100° – schon zwei Punkte standen dann
340     // an den Rändern und stiessen an die Nachbargruppe. So bleibt zwischen
341     // den drei Gruppen sichtbar Platz, auch wenn eine gut gefüllt ist.
342     let winkelBreiten = ringFormen.map(f => (f.d + FWERT_PUNKT_LUECKE) / ringRadius);
343     let reiheBreite = winkelBreiten.reduce((a, b) => a + b, 0);
344     let kante = mitte - reiheBreite / 2;
345     ringFormen.forEach((f, i) => {
346       let winkelPunkt = kante + winkelBreiten[i] / 2;
347       kante += winkelBreiten[i];
348       let x = cx + Math.cos(winkelPunkt) * ringRadius;
349       let y = cy + Math.sin(winkelPunkt) * ringRadius;
350       gesetzt.push({ x, y, typ: f.typ });
351       // Canvas-Pfad, siehe ACHTUNG oben.
352       drawingContext.fillStyle = `rgba(${f.rgb.r}, ${f.rgb.g}, ${f.rgb.b}, ${alphaSkala})`;
353       drawingContext.beginPath();
354       drawingContext.arc(x, y, f.d / 2, 0, TWO_PI);
355       drawingContext.fill();
356     });
357
358     ringRadius += FWERT_PUNKT_RING_ABSTAND;
359   }
360 });
361 pop();
362 return gesetzt;
363 }
364
365
366 // -----
367 // Demo-Kreisgrafik (Erklärung vor dem Zoom in Kapitel 1)
368 // -----
369
370 // Erfundene Zielwerte, keine Kapiteldaten. Drei klar unterscheidbare
371 // Bandgrößen (16/10/8).
372 //
373 // ACHTUNG alle drei Bänder müssen in dieselbe Richtung überwiegen, sonst füllt
374 // das negative Band eines anderen die untere Hälfte auf und die Ausbauchung
375 // ist nicht mehr zu sehen.
376 //
377 // ACHTUNG auch die pos-Werte müssen sich unterscheiden (8/6/4). Der
378 // Legendenaufbau zeigt je Kategorie nur die beiden Valenzhälften, und deren
379 // Radius hängt allein an pos bzw. neg – bei gleichem pos-Wert läge der neue
380 // Halbkreis genau auf dem vorigen, und der Schritt wäre nicht zu sehen.
381 const DEMO_BAND_COUNTS = {
382   gold_dunkel: { pos: 8, neg: 3, neutral: 3, unrated: 2 },
383   gold_mittel: { pos: 6, neg: 1, neutral: 2, unrated: 1 },
384   gold_hell: { pos: 4, neg: 2, neutral: 1, unrated: 1 },
385 };
386

```

```

387 // Umkehrung von valenzBucket(); unrated hat keinen Zahlenwert.
388 const DEMO_VALENZ = { pos: 1, neg: -1, neutral: 0, unrated: undefined };
389
390 // Ein F-Wert je Valenzgruppe, damit oben, unten und rechts je ein Punkt sitzt.
391 const DEMO_FWERTE = { pos: 'ort_loest_emotion_aus', neg: 'emotion_faerbt_raum', neutral:
  'koerper_als_sensor' };
392
393 // Die Demo steht allein auf der Karte und darf grösser sein als die Kreise
394 // entlang der Route. maxRadius ist der Vorgabewert von kreisRadius().
395 const DEMO_MAX_RADIUS = 100;
396 const DEMO_LABEL_ABSTAND = 45;
397
398 // Ruheplatz der beiden Icons: nebeneinander direkt über dem Kapitelregister,
399 // rechtsbündig zum Bildschirmrand. Links die Kreisgrafik (Legende), rechts
400 // der Projekttext.
401 const IKON_RAND_RECHTS = 46; // Mitte des rechten Icons zum Bildschirmrand
402 const IKON_ABSTAND_PAAR = 80; // Mitte zu Mitte der beiden Icons
403 const IKON_ABSTAND_REGISTER = 44; // Luft zwischen Icon-Mitte und Registerkante
404 const IKON_ABSTAND_OBEN_MIN = 50; // damit die Zeile nie oben herausfällt
405 const IKON_RADIUS_SKALA = 0.45;
406
407 // Ruhezustand schwarzgrau, erst der Zeiger holt das Gold hervor. Der Filter
408 // entsättigt die ganze Miniatur, statt jede Bandfarbe einzeln umzurechnen.
409 const IKON_GRAUFILTER = 'grayscale(1) brightness(0.45)';
410 const IKON_RUHE_FARBE = '#212B2E'; // wie die Kreis-Labels
411 const IKON_RUHE_FARBE_DUNKEL = '#E2E6E1'; // auf dem Schleier des Projekttextes
412 const IKON_GOLD = CATEGORY_COLORS.gold_mittel;
413
414 // Zweites Icon: ein Kreis mit waagrechten Zeilen – dieselbe Geste wie die
415 // Schraffur der Gesamtkreise, hier als Zeichen für Text. Wohnt hier, weil es
416 // sich Zeile und Treffertest mit dem Kreisgrafik-Icon teilt.
417 const IKON_TEXT_RADIUS = 23;
418 const IKON_TEXT_ZEILEN = [0.72, 0.94, 0.88, 0.56]; // Zeilenlängen, Anteil am Radius
419
420 // Lage der zuletzt gezeichneten Icons, für die Treffertests.
421 let letzteDemoLage = null;
422 let letzteProjekttextIkonLage = null;
423
424 // Höhe der Icon-Zeile, gemessen an der Oberkante des Kapitelregisters – das
425 // sitzt vertikal zentriert und wandert mit der Fensterhöhe.
426
427 // ACHTUNG getBoundingClientRect() erzwingt ein Layout. Deshalb gecacht und nur
428 // neu gelesen, wenn sich Fenstergrösse oder Registerhöhe geändert haben
429 // (Plan/Graph-Zeile fällt in der Übersicht weg, siehe draw()).
430 let ikonZeileCache = null;
431 function ikonZeileY() {
432   let schluessel = `${width}|${height}|${!!(modusZeile &&
modusZeile.classList.contains('versteckt'))}`;
433   if (!ikonZeileCache || ikonZeileCache.schluessel !== schluessel) {
434     let oben = kapitelRegister ? kapitelRegister.getBoundingClientRect().top : height * 0.14;
435     ikonZeileCache = { schluessel, y: Math.max(IKON_ABSTAND_OBEN_MIN, oben - IKON_ABSTAND_REGISTER)
};
436   }
437   return ikonZeileCache.y;
438 }
439
440 // Oberkante des Begleittexts. Aus seinen CSS-Werten gerechnet, nicht gemessen:
441 // getBoundingClientRect() je Frame wäre ein erzwungenes Layout, und die
442 // Zeilenzahl wechselt ohnehin mit jeder Stufe – gerechnet wird darum immer mit
443 // der längsten (vierzeiligen).
444 // ACHTUNG spiegelt .begleittext[data-demo-gruppe] in style.css (top 78 %,
445 // translateY(-50 %), font-size clamp(16px, 3vw, 30px), line-height 1.5).
446 // Ändert sich dort etwas, muss es hier nachgezogen werden, sonst rutscht die
447 // Legende in den Text.
448 const LEGENDE_TEXT_MITTE = 0.78;
449 const BEGLEITTEXT_ZEILENHOEHE = 1.5;

```



```

450 const BEGLEITTEXT_ZEILEN_MAX = 4;
451 function begleittextHalbeHoehe() {
452     let schrift = Math.min(30, Math.max(16, width * 0.03));
453     return schrift * BEGLEITTEXT_ZEILENHOEHE * BEGLEITTEXT_ZEILEN_MAX / 2;
454 }
455 function begleittextOben() {
456     return height * LEGENDE_TEXT_MITTE - begleittextHalbeHoehe();
457 }
458
459 // Luft über der Kopfzeile des Legendenaufbaus. Genauso viel, wie unter dem
460 // Begleittext frei bleibt – dadurch sitzt die ganze Komposition aus Titel,
461 // Kreis, Blockzeile und Text mittig im Fenster.
462 function legendenKopfraum() {
463     return height - (height * LEGENDE_TEXT_MITTE + begleittextHalbeHoehe());
464 }
465
466 // Höhe, die die Blockzeile über dem Begleittext belegt: Überschrift, drei
467 // Zeilen und die Luft darunter.
468 function blockZeileHoehe() {
469     return LEGENDE_TITEL_ABSTAND + 2 * LEGENDE_ZEILE + LEGENDE_BLOCK_LUFT;
470 }
471
472 // Radius des fertig aufgebauten Demo-Kreises bei Massstab 1. Hängt nur an
473 // DEMO_BAND_COUNTS, wird deshalb einmal berechnet und gemerkt.
474 let einheitsRadiusCache = null;
475 function demoEinheitsRadius() {
476     if (einheitsRadiusCache === null) {
477         einheitsRadiusCache = groessterKreisRadius(DEMO_BAND_COUNTS, DEMO_MAX_RADIUS, 1);
478     }
479     return einheitsRadiusCache;
480 }
481
482 // ikon 0 = Legendenplatz in der Bildmitte, 1 = Ruheplatz als Icon.
483 // Auf dem Legendenplatz steht der Kreis im Viewport zentriert. Der Massstab
484 // fällt auf kleinen Fenstern mit, sonst reicht der Platz unter dem Kreis nicht
485 // mehr für die beiden Legendenblöcke.
486 function demoKreisLage(ikon) {
487     // Über dem Begleittext liegen von oben nach unten: der Kreisgrößen-Text,
488     // der Kreis mit seinem Bogen und der Kategorienblock. Der Kreis nimmt, was
489     // dazwischen frei bleibt – kein fester Massstab, sondern der Zielradius
490     // geteilt durch den Radius, den derselbe Kreis bei Massstab 1 hätte.
491     let raum = begleittextOben() - legendenKopfraum() - blockZeileHoehe();
492     // Zweimal LEGENDE_BOGEN_ABSTAND: einmal für den Bogen selbst, einmal als
493     // Luft zwischen ihm und dem Kategorienblock darunter.
494     let ziel = Math.min(width * 0.26,
495         Math.max(40, (raum - LEGENDE_ECKE - 2 * LEGENDE_BOGEN_ABSTAND) / 2));
496     return {
497         cx: lerp(width * 0.5, width - IKON_RAND_RECHTS - IKON_ABSTAND_PAAR, ikon),
498         cy: lerp(legendenKopfraum() + LEGENDE_ECKE + ziel, ikonZeileY(), ikon),
499         skala: lerp(ziel / demoEinheitsRadius(), IKON_RADIUS_SKALA, ikon),
500     };
501 }
502
503 // alphaSkala blendet mit dem Kreisgrafik-Icon ein, hover holt das Gold.
504 // aufDunkel dreht den Ruheton hell, solange der Projekttext offen ist.
505 function zeichneProjekttextIkon(alphaSkala, hover, aufDunkel = false) {
506     letzteProjekttextIkonLage = null;
507     if (alphaSkala <= 0.01) return;
508     let cx = width - IKON_RAND_RECHTS, cy = ikonZeileY(), r = IKON_TEXT_RADIUS;
509
510     push(); // schreibt strokeStyle direkt, siehe ACHTUNG oben
511     drawingContext.globalAlpha = alphaSkala;
512     drawingContext.strokeStyle = hover ? IKON_GOLD
513         : (aufDunkel ? IKON_RUHE_FARBE_DUNKEL : IKON_RUHE_FARBE);
514     drawingContext.lineWidth = 1.8;
515     drawingContext.beginPath();

```



```

516   drawingContext.arc(cx, cy, r, 0, TWO_PI);
517   drawingContext.stroke();
518
519   let abstand = r * 0.4;
520   let obenY = cy - abstand * (IKON_TEXT_ZEILEN.length - 1) / 2;
521   IKON_TEXT_ZEILEN.forEach((anteil, i) => {
522     let halb = r * anteil * 0.72;
523     let y = obenY + i * abstand;
524     drawingContext.beginPath();
525     drawingContext.moveTo(cx - halb, y);
526     drawingContext.lineTo(cx + halb, y);
527     drawingContext.stroke();
528   });
529   pop();
530
531   letzteProjekttextIkonLage = { cx, cy, r, alpha: alphaSkala };
532 }
533
534 function projekttextIkonGetroffen(mx, my) {
535   let l = letzteProjekttextIkonLage;
536   if (!l || l.alpha <= 0.5) return false; // erst am Ruheplatz anklickbar
537   return dist(mx, my, l.cx, l.cy) <= l.r + 8;
538 }
539
540 // Aufdeck-Reihenfolge: reihum durch die Bänder, damit alle drei gemeinsam
541 // wachsen. Der Scroll-Fortschritt deckt davon einen Präfix auf, genau wie
542 // annIndex bei den echten Kreisen.
543 function baueDemoAnnotationen() {
544   let vorraete = KREIS_KATEGORIEN.map(kat =>
545     Object.keys(DEMO_VALENZ).flatMap(bucket =>
546       Array.from({ length: DEMO_BAND_COUNTS[kat.key][bucket] || 0 },
547         () => ({ category: kat.key, valenz: DEMO_VALENZ[bucket] })));
548
549   let reihenfolge = [];
550   while (vorraete.some(v => v.length)) {
551     vorraete.forEach(v => { if (v.length) reihenfolge.push(v.shift()); });
552   }
553   // Die jeweils erste Annotation einer Valenzgruppe trägt deren F-Wert.
554   Object.keys(DEMO_FWERTE).forEach(bucket => {
555     let treffer = reihenfolge.find(a => a.valenz === DEMO_VALENZ[bucket]);
556     if (treffer) { treffer.hasFwert = true; treffer.fWertType = DEMO_FWERTE[bucket]; }
557   });
558   return reihenfolge;
559 }
560
561 const DEMO_ANNOTATIONEN = baueDemoAnnotationen();
562
563 // Beschriftungen am Kreis, in drei Gruppen passend zu den Erklärungstexten in
564 // index.html (data-demo-gruppe): Grösse, Wölbung, F-Wert-Punkte. Alle Labels
565 // stehen auf einer Seite, die Hilfslinien zeigen auf die Stelle, die der Text
566 // meint. Gilt für die Demo-Grafik wie für jeden echten Ortskreis – was ein
567 // Kreis nicht zeigt (fehlendes Band, fehlende Valenz), bekommt kein Label.
568 function kreisBeschriftungen(cx, cy, bandCounts, aussen, skala, maxRadius, gruppenAlpha,
569   fwertPunkte, links) {
570   let labelX = links ? cx - aussen - DEMO_LABEL_ABSTAND : cx + aussen + DEMO_LABEL_ABSTAND;
571   let eintrag = (gruppe, text, radius, winkel) => {
572     let ankerY = cy + Math.sin(winkel) * radius;
573     return {
574       ankerX: cx + Math.cos(winkel) * radius, ankerY,
575       x: labelX, y: ankerY, text, farbe: null,
576       hilfslinie: true, links, alpha: gruppenAlpha[gruppe],
577     };
578   };
579   // Fächer über eine Kreisseite, damit sich die Hilfslinien nicht kreuzen.
580   let winkelBand = [-0.95, -0.35, 0.3];
581   let bandAnzahl = key => {

```

```

581     let b = bandCounts[key] || {};
582     return (b.neg || 0) + (b.pos || 0) + (b.neutral || 0) + (b.unrated || 0);
583 };
584 let valenzAnzahl = bucket => KREIS_KATEGORIEN.reduce(
585   (n, kat) => n + ((bandCounts[kat.key] || {})[bucket] || 0), 0);
586
587 let labels = [];
588 KREIS_KATEGORIEN.forEach((kat, i) => {
589   let n = bandAnzahl(kat.key);
590   if (n > 0) labels.push(eintrag(0, CATEGORY_LABELS[kat.key],
591     kreisRadius(n, maxRadius) * skala, winkelBand[i]));
592 });
593 // Einzeilig, deshalb die ersten drei der fünf PDF-Zeilen zusammengezogen.
594 labels.push(eintrag(0, LEGENDE_KREISGROESSE.slice(0, 3).join(' '), aussen, 0.85));
595
596 if (valenzAnzahl('pos') > 0) labels.push(eintrag(1, LEGENDE_VALENZ.pos, aussen * 0.7, -PI / 3));
597 if (valenzAnzahl('neg') > 0) labels.push(eintrag(1, LEGENDE_VALENZ.neg, aussen * 0.7, PI / 3));
598 // Nicht auf Winkel 0: dort sitzt der neutrale F-Wert-Punkt, die Hilfslinie
599 // liefere genau durch ihn. Für neutrale Gefühle kennt das PDF keine Klammer –
600 // der Wortlaut setzt deshalb das Muster der beiden anderen fort.
601 if (valenzAnzahl('neutral') > 0) labels.push(eintrag(1, 'Anteil neutraler Gefühle', aussen * 0.45,
602   0.32));
603
604 // Je F-Wert-Typ ein Label, angeheftet an einen wirklich gesetzten Punkt
605 // dieses Typs – die Punktgrösse ist es, die den Typ unterscheidet.
606 let schonBeschriftet = new Set();
607 (fwertPunkte || []).forEach(p => {
608   if (!p.typ || schonBeschriftet.has(p.typ)) return;
609   schonBeschriftet.add(p.typ);
610   labels.push({
611     ankerX: p.x, ankerY: p.y, x: labelX, y: p.y,
612     text: FWERT_LABELS[p.typ], farbe: null,
613     hilfslinie: true, links, alpha: gruppenAlpha[2],
614   });
615 });
616
617 // zeichneKreisLabels weicht nur nach unten aus: liegt der Kreis tief im
618 // Bild, muss der Stapel vorher hoch, sonst fällt er unten heraus.
619 let stapel = labels.length * (LABEL_HOEHE + LABEL_ABSTAND);
620 let obenRaus = LABEL_HOEHE;
621 let untenRaus = Math.max(obenRaus, height - stapel);
622 labels.forEach(l => l.y = constrain(l.y, obenRaus, untenRaus));
623 return labels;
624 }
625
626 // -----
627 // Legendenaufbau (Onboarding)
628 // -----
629
630 const LEGENDE_ZEILE = 22; // Zeilenabstand in den beiden Blöcken
631 const LEGENDE_TITEL_ABSTAND = 26; // Überschrift zur ersten Zeile
632 const LEGENDE_FELD = 15; // Kantenlänge der Kategorienfelder
633 const LEGENDE_MARKE_SPALTE = 26; // Feld- bzw. Punktspalte zum Text
634 const LEGENDE_BLOCK_LUECKE = 40; // Luft zwischen den beiden Blöcken
635 const LEGENDE_BLOCK_LUFT = 22; // Blockzeile zur Oberkante des Begleittexts
636 const LEGENDE_RAND_LINKS = 46; // Titel und linker Block zum Fensterrand
637 const LEGENDE_TITEL_OBEN = 84; // Mitte der Titelzeile – unterhalb des
638 // Schliessknopfs der Erklärungs-Ebene
639 // (.einblender-schliessen, 28px + 36px)
640 const LEGENDE_TEXTZEILE = 17; // Zeilenabstand im Kreisgrößen-Block
641 const LEGENDE_TEXT_ABSTAND = 60; // Kreisrand zum Text rechts
642 const LEGENDE_ECKE = 46; // Höhe des Knicks über dem Kreisscheitel
643 const LEGENDE_KLAMMER_TIEFE = 90; // Wie weit die Valenzklammern nach links greifen
644 const LEGENDE_BOGEN_ABSTAND = 30; // Bogenradius über dem Kreisrand
645 const LEGENDE_RING_RADIUS = 7; // offene Ringe auf dem Bogen

```

```

646 const LEGENDE_MITTELPUNKT = 4;      // dunkler Punkt in der Kreismitte, wie in zeichneKreiseFuerRun
647 const LEGENDE_STRICHEL = [3, 4];    // Strichelmass von Klammern und Bogen
648 const LEGENDE_SICHTBAR = 0.002;     // darunter lohnt das Zeichnen nicht
649 const LEGENDE_TINTE_RGB = hexZuRgb(IKON_RUHE_FARBE);
650
651 // Zeilen des Blocks «Körper und Raum»
652 const LEGENDE_FWERT_ZEILEN = Object.keys(FWERT_PUNKTGROESSE)
653   .sort((a, b) => FWERT_PUNKTGROESSE[b] - FWERT_PUNKTGROESSE[a])
654   .map(typ => ({ text: FWERT_LABELS[typ], punkt: FWERT_PUNKT_DURCHMESSER[FWERT_PUNKTGROESSE[typ]]
655   }));
656
657 // Der Kreis differenziert sich mit der Legende: erst
658 // gestreift, dann mit Valenzhälften, dann mit den drei Bändern.
659 function stufenBandCounts(bandCounts, mitKategorie, valenzen) {
660   // Was noch keine Stufe benannt hat, bleibt Schraffur: der Kreis wächst nicht
661   // mehr, es füllt sich nur, was schon erklärt ist.
662   let aufteilen = bc => {
663     let raus = { neg: 0, pos: 0, neutral: 0, unrated: 0 };
664     Object.keys(raus).forEach(bucket => {
665       let n = bc[bucket] || 0;
666       if (bucket !== 'unrated' && valenzen.includes(bucket)) raus[bucket] = n;
667       else raus.unrated += n;
668     });
669     return raus;
670   };
671
672   // Ohne Kategorie: ein einziges Band. Die Mengen werden
673   // auf das grösste Band heruntergerechnet statt summiert – die Summe ergäbe
674   // einen grösseren Kreis, und der Aussenradius soll über alle Stufen stehen
675   // bleiben. gold_mittel, weil zeichneKreiseFuerRun genau dieses Band deckend
676   // zeichnet statt im Multiply; ein einzelnes Band soll nicht nachdunkeln.
677   /*if (!mitKategorie) {
678     let summe = { neg: 0, pos: 0, neutral: 0, unrated: 0 };
679     let groesstesBand = 0;
680     KREIS_KATEGORIEN.forEach(kat => {
681       let b = bandCounts[kat.key] || {};
682       let n = 0;
683       Object.keys(summe).forEach(bucket => {
684         summe[bucket] += b[bucket] || 0;
685         n += b[bucket] || 0;
686       });
687       groesstesBand = Math.max(groesstesBand, n);
688     });
689     let gesamt = summe.neg + summe.pos + summe.neutral + summe.unrated;
690     if (!gesamt) return {};
691     let f = groesstesBand / gesamt;
692     return { gold_mittel: aufteilen({
693       neg: summe.neg * f, pos: summe.pos * f,
694       neutral: summe.neutral * f, unrated: summe.unrated * f,
695     }) };
696   }*/
697
698   // Mit Kategorie: das erste Band echt. Die weiteren kommen nicht hierher,
699   // sondern als blosse Valenzhälften dazu (zeichneKategorieHaelften) – bei
700   // DEMO_BAND_COUNTS ist das erste zugleich das grösste, der Aussenradius
701   // bleibt dadurch über alle Stufen stehen.
702   let erste = KREIS_KATEGORIEN[0];
703   return bandCounts[erste.key] ? { [erste.key]: aufteilen(bandCounts[erste.key]) } : {};
704 }
705
706 // Ein weiteres Band bringt nur seine beiden Valenzhälften mit: kein
707 // Schraffurkreis, keine neutrale Fläche. Ein voller Ring je Kategorie machte
708 // die Mitte zu dicht – mit drei Bändern lägen dort acht Kreise übereinander.
709 // Dieselbe Regel gilt über VALENZEN auch für das erste Band.
710 // Winkel wie in zeichneKreiseFuerRun (winkel PI): positiv oben, negativ unten.
711 function zeichneKategorieHaelften(cx, cy, bc, kat, alphaSkala, skala) {

```

```

711     if (!bc) return;
712     let negR = kreisRadius(bc.neg || 0, DEMO_MAX_RADIUS) * skala;
713     let posR = kreisRadius(bc.pos || 0, DEMO_MAX_RADIUS) * skala;
714     if (negR > 0) zeichneHalbkreis(cx, cy, negR, HALF_PI, kat.farbe, alphaSkala);
715     if (posR > 0) zeichneHalbkreis(cx, cy, posR, -HALF_PI, kat.farbe, alphaSkala);
716 }
717
718 // Ein Zustand des Demo-Kreises: das erste Band ganz, jedes weitere nur mit
719 // seinen Valenzhälften. kategorien 0 = noch gar keine, dann ein tonloses Band.
720 function zeichneDemoStufe(cx, cy, bandCounts, kategorien, valenzen, alphaSkala, skala) {
721     if (alphaSkala <= LEGENDE_SICHTBAR) return;
722     zeichneKreiseFuerRun(cx, cy, stufenBandCounts(bandCounts, kategorien > 0, valenzen),
723         alphaSkala, PI, skala, DEMO_MAX_RADIUS);
724     KREIS_KATEGORIEN.slice(1, kategorien).forEach(kat =>
725         zeichneKategorieHaelften(cx, cy, bandCounts[kat.key], kat, alphaSkala, skala));
726 }
727
728 // Schrift aller Canvas-Beschriftungen: wie .annotation-tag (var(--sans)),
729 // font-weight 700. Messen und Zeichnen müssen dieselbe nehmen, sonst stimmen
730 // Blockbreite und Titelkante nicht mehr mit dem überein, was danebensteht –
731 // deshalb an einer Stelle.
732 function beschriftungsSchrift(groesse) {
733     textFont(SCHRIFT_SANS);
734     textSize(groesse);
735     textStyle(BOLD);
736 }
737
738 // Gestrichelte Linien der Legende. Kapselt das setLineDash-Paar, das sonst in
739 // jeder der drei Zeichenroutinen stünde.
740 function legendenStrich(farbe, alpha, zeichnen) {
741     push();
742     noFill();
743     stroke(farbe.r, farbe.g, farbe.b, 255 * alpha);
744     strokeWeight(0.9);
745     drawingContext.setLineDash(LEGENDE_STRICHEL);
746     zeichnen();
747     drawingContext.setLineDash([]);
748     pop();
749 }
750
751 // Breite eines Texts in der Beschriftungsschrift. Kapselt das push/pop, damit
752 // Messen und Zeichnen denselben Zustand sehen.
753 function beschriftungsBreite(text, groesse = LABEL_GROESSE) {
754     push();
755     beschriftungsSchrift(groesse);
756     let breite = textWidth(text);
757     pop();
758     return breite;
759 }
760
761 // Kopf der Legende, im PDF auf jeder Seite gleich. Lage gibt der Aufrufer vor – im Legendenaufbau
762 // steht er auf der Zeile von
763 // «Anteil positiver Gefühle», in der Erklärungs-Ebene unter dem Schliessknopf.
764 function zeichneLegendenTitel(x, y, alpha) {
765     if (alpha <= LEGENDE_SICHTBAR) return;
766     push();
767     noStroke();
768     textAlign(LEFT, CENTER);
769     beschriftungsSchrift(LABEL_GROESSE);
770     fill(LEGENDE_TINTE_RGB.r, LEGENDE_TINTE_RGB.g, LEGENDE_TINTE_RGB.b, 255 * alpha);
771     text(LEGENDE_TITEL, x, y);
772     // Zurückgenommen wie im PDF: der Untertitel benennt nur, was das Bild ist.
773     // Gleiche Grösse, unterschieden über Gewicht und Deckkraft.
774     textStyle(NORMAL);
775     fill(LEGENDE_TINTE_RGB.r, LEGENDE_TINTE_RGB.g, LEGENDE_TINTE_RGB.b, 140 * alpha);
776     text(LEGENDE_UNTERTITEL, x, y + LEGENDE_TEXTZEILE);

```

```

776     pop();
777 }
778
779 // Schritt 1: der Kreisgrößen-Text rechts oben, angebunden mit einem Knick
780 // über dem Kreisscheitel. Steht ausserhalb von zeichneKreisLabels, weil er
781 // mehrzeilig ist – dort bekäme jede Zeile eine eigene Hilfslinie.
782 function zeichneKreisgroessenBlock(cx, cy, aussen, alpha) {
783     let x = cx + aussen + LEGENDE_TEXT_ABSTAND;
784     let ecke = cy - aussen - LEGENDE_ECKE;
785     legendenStrich(LEGENDE_TINTE_RGB, alpha, () => {
786         line(cx, cy - aussen, cx, ecke);
787         line(cx, ecke, x - 8, ecke);
788     });
789     push();
790     noStroke();
791     textAlign(LEFT, CENTER);
792     beschriftungsSchrift(LABEL_GROESSE);
793     LEGENDE_KREISGROESSE.forEach((zeile, i) => {
794         // Die ersten drei Zeilen benennen die Regel, die letzten beiden erläutern
795         // sie – im PDF derselbe Gewichtswechsel.
796         textStyle(i < 3 ? BOLD : NORMAL);
797         fill(LEGENDE_TINTE_RGB.r, LEGENDE_TINTE_RGB.g, LEGENDE_TINTE_RGB.b, 255 * alpha);
798         text(zeile, x, ecke + i * LEGENDE_TEXTZEILE);
799     });
800     pop();
801 }
802
803 // Schritte 2 und 3: je eine nach rechts offene Klammer über einer Kreishälfte.
804 // richtung -1 = obere (positiv), +1 = untere (negativ). Die Mittellinie
805 // gehört beiden und wird von demoLegende einmal gezogen – zweimal übereinander
806 // dunkelte sie nach, solange sich die beiden Schritte überblenden.
807 function zeichneValenzKlammer(cx, cy, aussen, richtung, alpha) {
808     let links = cx - aussen - LEGENDE_KLAMMER_TIEFE;
809     let y = cy + richtung * aussen;
810     legendenStrich(LEGENDE_TINTE_RGB, alpha, () => {
811         line(links, y, cx, y);
812         line(links, y, links, cy);
813     });
814 }
815
816 // Breite eines Blocks: Markenspalte plus längste Zeile. Nur der rechte Block
817 // braucht sie, um am Fensterrand zu enden.
818 function legendenBlockBreite(titel, zeilen) {
819     return Math.max(beschriftungsBreite(titel),
820         ...zeilen.map(z => LEGENDE_MARKE_SPALTE + beschriftungsBreite(z.text)));
821 }
822
823 // Schritte 4 und 5: ein Block mit Überschrift und Zeilen. Eine Zeile trägt
824 // entweder ein Farbfeld (Kategorien) oder einen Punkt (Körper und Raum).
825 // alpha gilt für die Überschrift; eine Zeile kann mit z.alpha ihre eigene
826 // mitbringen – die Kategorien kommen einzeln, mit dem Band, das sie benennen.
827 function zeichneLegendenBlock(x, y, titel, zeilen, alpha) {
828     if (alpha <= LEGENDE_SICHTBAR) return;
829     push();
830     noStroke();
831     textAlign(LEFT, CENTER);
832     beschriftungsSchrift(LABEL_GROESSE);
833     fill(LEGENDE_TINTE_RGB.r, LEGENDE_TINTE_RGB.g, LEGENDE_TINTE_RGB.b, 255 * alpha);
834     text(titel, x, y);
835     zeilen.forEach((z, i) => {
836         let a = z.alpha === undefined ? alpha : z.alpha;
837         if (a <= LEGENDE_SICHTBAR) return;
838         let zy = y + LEGENDE_TITEL_ABSTAND + i * LEGENDE_ZEILE;
839         if (z.feld) {
840             fill(z.feld[0], z.feld[1], z.feld[2], 255 * a);
841             rect(x, zy - LEGENDE_FELD / 2, LEGENDE_FELD, LEGENDE_FELD);

```

```

842     } else {
843         fill(FWERT_COLOR_RGB.r, FWERT_COLOR_RGB.g, FWERT_COLOR_RGB.b, 255 * a);
844         circle(x + LEGENDE_FELD / 2, zy, z.punkt);
845     }
846     fill(LEGENDE_TINTE_RGB.r, LEGENDE_TINTE_RGB.g, LEGENDE_TINTE_RGB.b, 255 * a);
847     text(z.text, x + LEGENDE_MARKE_SPALTE, zy);
848 });
849 pop();
850 }
851
852 // Stufen 6 bis 8 (PDF-Seiten 7 bis 9): je ein gestrichelter Bogenabschnitt mit
853 // einem offenen Ring in seiner Mitte – genau dort sitzen die F-Wert-Punkte
854 // (FWERT_GRUPPEN_WINKEL). Ein Bogen je Gruppe statt eines vollen Kreises: die
855 // Lücken zeigen, dass die Punkte in drei Abschnitten liegen, nicht rundum.
856 // alphas nennt je Valenzgruppe die Deckkraft; gibt die sichtbaren Ringe für
857 // die Beschriftung zurück.
858 function zeichneWahrnehmungsbogen(cx, cy, aussen, alphas) {
859     let r = aussen + LEGENDE_BOGEN_ABSTAND;
860     let ringe = Object.entries(FWERT_GRUPPEN_WINKEL)
861         .map(([bucket, winkel]) => ({
862             bucket, winkel, alpha: alphas[bucket] || 0,
863             x: cx + Math.cos(winkel) * r, y: cy + Math.sin(winkel) * r,
864         })))
865         .filter(ring => ring.alpha > LEGENDE_SICHTBAR);
866     ringe.forEach(ring => {
867         legendenStrich(FWERT_COLOR_RGB, ring.alpha, () => arc(cx, cy, r * 2, r * 2,
868             ring.winkel - FWERT_GRUPPEN_SPANNE / 2, ring.winkel + FWERT_GRUPPEN_SPANNE / 2, OPEN));
869         push();
870         noFill();
871         stroke(FWERT_COLOR_RGB.r, FWERT_COLOR_RGB.g, FWERT_COLOR_RGB.b, 255 * ring.alpha);
872         strokeWeight(1.4);
873         circle(ring.x, ring.y, LEGENDE_RING_RADIUS * 2);
874         pop();
875     });
876     return ringe;
877 }
878
879 // Baut die Legende um den Demo-Kreis auf. Zeichnet alles mit eigener Geometrie
880 // selbst und gibt die Beschriftungen als Kandidaten zurück – zeichneKreisLabels
881 // wird genau einmal aufgerufen, sonst weichen sich die Labels nicht aus.
882 function demoLegende(cx, cy, aussen, gruppenAlpha, kreisDa) {
883     let [aOrt, aGrosse, aPos, aNeg, aMittel, aHell, aWpos, aWneg, aWneutral] = gruppenAlpha;
884     let labels = [];
885     let rechts = cx + aussen + LEGENDE_TEXT_ABSTAND;
886     let linksX = cx - aussen - LEGENDE_KLAMMER_TIEFE - 10;
887     let hinzu = (alpha, links, ankerX, ankerY, text, farbe, hilfslinie) => labels.push({
888         ankerX, ankerY, x: links ? linksX : rechts, y: ankerY,
889         text, farbe: farbe || null, hilfslinie, links, alpha,
890     });
891
892     // Schritt 0 (PDF-Seite 1): der Mittelpunkt und sein Name stehen da, bevor
893     // der Kreis wächst. Sobald er da ist, zeichnet zeichneKreiseFuerRun den
894     // Punkt selbst – hier bliebe er sonst ein zweites Mal darüberliegen.
895     // Über «Positive Wahrnehmung» und linksbündig zu dieser Beschriftung: deren
896     // Zeile ergibt sich aus dem Winkel und Radius, auf dem
897     // zeichneWahrnehmungsbogen ihren Ring setzt. Drei Zeilen darüber – zwei
898     // trägt der Kopf selbst (Titel und Unterzeile), die dritte ist der Abstand
899     // zur Beschriftung. Bei zwei Zeilen blieben darunter nur 9 px.
900     let obersteZeile = cy + Math.sin(FWERT_GRUPPEN_WINKEL.pos) * (aussen + LEGENDE_BOGEN_ABSTAND);
901     zeichneLegendenTitel(
902         Math.max(LEGENDE_RAND_LINKS, linksX - beschriftungsBreite(WAHRNEHMUNG_LABELS.pos)),
903         obersteZeile - 3 * LEGENDE_TEXTZEILE, aOrt);
904
905     if (aOrt > LEGENDE_SICHTBAR) {
906         if (!kreisDa) {
907             push();

```



```

908     noStroke();
909     fill(LEGENDE_TINTE_RGB.r, LEGENDE_TINTE_RGB.g, LEGENDE_TINTE_RGB.b, 255 * a0rt);
910     circle(cx, cy, LEGENDE_MITTELPUNKT * 2);
911     pop();
912 }
913 hinzu(a0rt, false, cx, cy, LEGENDE_ORTSBESCHRIFTUNG, null, true);
914 }
915
916 if (aGroesse > LEGENDE_SICHTBAR) zeichneKreisgroessenBlock(cx, cy, aussen, aGroesse);
917
918 // Schritte 2 und 3: erst die obere Hälfte, dann die untere (PDF-Seiten 3
919 // und 4). Die Klammer ist die Hilfslinie; eine zweite liefe daneben her.
920 let klammer = cx - aussen - LEGENDE_KLAMMER_TIEFE;
921 let valenz = Math.max(aPos, aNeg);
922 if (valenz > LEGENDE_SICHTBAR) {
923     legendenStrich(LEGENDE_TINTE_RGB, valenz, () => line(klammer, cy, cx, cy));
924 }
925 if (aPos > LEGENDE_SICHTBAR) {
926     zeichneValenzKlammer(cx, cy, aussen, -1, aPos);
927     hinzu(aPos, true, klammer, cy - aussen / 2, LEGENDE_VALENZ.pos, null, false);
928 }
929 if (aNeg > LEGENDE_SICHTBAR) {
930     zeichneValenzKlammer(cx, cy, aussen, 1, aNeg);
931     hinzu(aNeg, true, klammer, cy + aussen / 2, LEGENDE_VALENZ.neg, null, false);
932 }
933
934 // Stufen 2, 4 und 5 (PDF-Seiten 3, 5 und 6): jede Bandzeile kommt mit dem
935 // Band, das sie benennt. Stufe 2 bringt zugleich die Überschrift mit.
936 let kategorieZeilen = KREIS_KATEGORIEN.map((kat, i) => ({
937     text: CATEGORY_LABELS[kat.key], feld: kat.farbe,
938     alpha: [aPos, aMittel, aHell][i],
939 }));
940 // Stufe 6 (PDF-Seite 7) bringt den zweiten Block ganz, mit allen drei Zeilen.
941 let fwertZeilen = LEGENDE_FWERT_ZEILEN.map(z => ({ ...z, alpha: aWpos }));
942
943 // Beide Blöcke nebeneinander auf einer Zeile zwischen Kreis und Begleittext
944 // – dort, wo die Bänder und Punkte liegen, die sie benennen. Die Breite des
945 // zweiten wird auch dann schon eingerechnet, wenn er noch gar nicht sichtbar
946 // ist; sonst spränge der erste zur Seite, sobald der zweite dazukommt.
947 let bKat = legendenBlockBreite(LEGENDE_BLOCK_TITEL.kategorien, kategorieZeilen);
948 let bFwert = legendenBlockBreite(LEGENDE_BLOCK_TITEL.fwerte, fwertZeilen);
949 let blockY = begleittextOben() - blockZeileHoehe();
950 let blockX = (width - (bKat + LEGENDE_BLOCK_LUECKE + bFwert)) / 2;
951 zeichneLegendenBlock(blockX, blockY, LEGENDE_BLOCK_TITEL.kategorien, kategorieZeilen, aPos);
952 zeichneLegendenBlock(blockX + bKat + LEGENDE_BLOCK_LUECKE, blockY,
953     LEGENDE_BLOCK_TITEL.fwerte, fwertZeilen, aWpos);
954
955 // Stufen 6 bis 8: ein Bogenabschnitt je Wahrnehmung. neutral sitzt rechts,
956 // positiv und negativ auf der Klammerseite.
957 zeichneWahrnehmungsbogen(cx, cy, aussen,
958     { pos: aWpos, neg: aWneg, neutral: aWneutral }).forEach(ring =>
959     hinzu(ring.alpha, ring.bucket !== 'neutral', ring.x, ring.y,
960         WAHRNEHMUNG_LABELS[ring.bucket],
961         `rgba(${FWERT_COLOR_RGB.r}, ${FWERT_COLOR_RGB.g}, ${FWERT_COLOR_RGB.b}, ${ring.alpha})`,
962         true));
963 return labels;
964 }
965
966 // fortschritt 0..1 deckt die erfundenen Annotationen auf, alphaSkala blendet
967 // die ganze Grafik, schritte die neun Legendenstufen (monoton, siehe draw()),
968 // schleier blendet allein die Beschriftungen wieder aus, ikon 0..1 schiebt
969 // alles vom Legendenplatz auf den Icon-Platz oben rechts.
970 function zeichneDemoKreisgrafik(fortschritt, alphaSkala, schritte, schleier, ikon = 0, grau = false)
971 {
972     letzteDemoLage = null;
973     if (alphaSkala <= 0) return;

```



```

972 let [, , aPos, aNeg, aMittel, aHell] = schritte;
973 let sichtbar = DEMO_ANNOTATIONEN.slice(0, Math.round(fortschritt * DEMO_ANNOTATIONEN.length));
974 let { cx, cy, skala } = demoKreisLage(ikon);
975 // Endradius, nicht der des halb aufgedeckten Kreises: sonst wanderte die
976 // ganze Legende mit, während der Kreis heranwächst.
977 let aussen = demoEinheitsRadius() * skala;
978
979 push(); // trägt den Graufilter; die inneren push/pop erben ihn
980 if (grau) drawingContext.filter = IKON_GRAUFILTER;
981 if (sichtbar.length) {
982   let bandCounts = zaehleBandCounts(sichtbar);
983   // winkel PI wie alle anderen Ansichten: positiv oben, negativ unten.
984   // Höchstens zwei Stufen überblenden sich, deshalb reicht das Gegenpaar.
985   // Nur pos und neg: neutrale Nennungen zeichnet zeichneKreiseFuerRun als
986   // Vollkreis, und der legte sich beim Schritt «Anteil negativer Gefühle»
987   // als geschlossene Scheibe über beide Hälften. Sie bleiben deshalb in der
988   // Schraffur – der Aussenradius zählt sie ohnehin mit, er bleibt stehen.
989   const VALENZEN = ['pos', 'neg'];
990   let stufe = (kategorien, valenzen, a) => zeichneDemoStufe(cx, cy, bandCounts,
991     kategorien, valenzen, alphaSkala * a, skala);
992   stufe(0, [], 1 - aPos);
993   stufe(1, ['pos'], aPos * (1 - aNeg));
994   stufe(1, VALENZEN, aNeg * (1 - aMittel));
995   stufe(2, VALENZEN, aMittel * (1 - aHell));
996   stufe(3, VALENZEN, aHell);
997   // Keine gefüllten F-Wert-Punkte in der Demo: dort stehen die offenen Ringe
998   // des Wahrnehmungsbogens für dieselbe Sache und zeigen sie deutlicher.
999 }
1000 zeichneKreisLabels(demoLegende(cx, cy, aussen,
1001   schritte.map(a => a * alphaSkala * schleier), sichtbar.length > 0));
1002 pop();
1003 letzteDemoLage = { cx, cy, r: aussen, ikon, alpha: alphaSkala };
1004 }
1005
1006 // Trefferfläche des Icons – nur wenn es auch wirklich als Icon dasteht und
1007 // nicht gerade noch auf dem Weg dorthin ist.
1008 function demoIkonGetroffen(mx, my) {
1009   let l = letzteDemoLage;
1010   if (!l || l.ikon < 0.99 || l.alpha <= 0.01) return false;
1011   return dist(mx, my, l.cx, l.cy) <= l.r + FWERT_PUNKT_RAND_ABSTAND + 8;
1012 }
1013
1014 // -----
1015 // Erklärungs-Ebene (Klick aufs Icon)
1016 // -----
1017
1018 // Heller Schleier über der ganzen Ansicht; die Kreisgrafik bleibt darunter
1019 // erkennbar, tritt aber hinter die Beschriftungen zurück.
1020 const ERKLAERUNG_SCHLEIER = '#E2E6E1';
1021 const ERKLAERUNG_SCHLEIER_ALPHA = 0.6;
1022 const ERKLAERUNG_LABEL_BREITE = 360; // Platzbedarf des längsten Labels
1023
1024 // Legt den Schleier über die Ansicht und beschriftet den grössten Kreis des
1025 // Frames mit allen drei Erklärungen zugleich. Steht keiner im Bild
1026 // (Übersichts- und Schlussakt), erklärt die Demo-Grafik an ihrem alten Platz.
1027 // Deckt die ganze Fläche ein. Auch der Projekttext-Einblender nutzt sie,
1028 // nur mit dunkler Farbe (siehe draw() in sketch.js).
1029 function zeichneSchleier(farbe, alphaSkala) {
1030   push();
1031   noStroke();
1032   drawingContext.globalAlpha = alphaSkala;
1033   drawingContext.fillStyle = farbe;
1034   drawingContext.fillRect(0, 0, width, height);
1035   pop();
1036 }
1037

```

```
1038 function zeichneKreisErklaerung() {
1039     zeichneSchleier(ERKLAERUNG_SCHLEIER, ERKLAERUNG_SCHLEIER_ALPHA);
1040
1041     // Ein angeschnittener Kreis taugt nicht als Bezug: seine Beschriftungen
1042     // zeigten auf Stellen ausserhalb des Bildes. Nur wenn gar keiner ganz
1043     // drinsteht, zählt wieder das ganze Feld.
1044     let groesster = (liste) => liste.reduce((bisher, k) =>
1045         !bisher || k.radius > bisher.radius ? k : bisher, null);
1046     let imBild = kreiseDiesesFrames();
1047     let ganzImBild = imBild.filter(k =>
1048         k.cx - k.radius > 0 && k.cx + k.radius < width &&
1049         k.cy - k.radius > 0 && k.cy + k.radius < height);
1050     let kreis = groesster(ganzImBild) || groesster(imBild);
1051     if (!kreis) {
1052         zeichneDemoKreisgrafik(1, 1, Array(9).fill(1), 1, 0); // bringt den Kopf mit
1053         return;
1054     }
1055     zeichneLegendenTitel(LEGENDE_RAND_LINKS, LEGENDE_TITEL_OBEN, 1);
1056     // Passen die Labels rechts nicht mehr aufs Bild, klappen sie nach links.
1057     let links = kreis.cx + kreis.radius + DEMO_LABEL_ABSTAND + ERKLAERUNG_LABEL_BREITE > width;
1058     zeichneKreisLabels(kreisBeschriftungen(kreis.cx, kreis.cy, kreis.bandCounts, kreis.radius,
1059         kreis.skala, kreis.maxRadius, [1, 1, 1], kreis.fwertPunkte, links));
1060 }
1061
1062 // --- Export -----
1063 // Dreizehn Namen. Leser: docs/architektur.md.
1064 window.leereBandCounts = leereBandCounts;
1065 window.zeichneSchleier = zeichneSchleier;
1066 window.zeichneProjekttextIkon = zeichneProjekttextIkon;
1067 window.projekttextIkonGetroffen = projekttextIkonGetroffen;
1068 window.zeichneKreisLabels = zeichneKreisLabels;
1069 window.zeichneKreiseOrtRuns = zeichneKreiseOrtRuns;
1070 window.zeichneKreiseFuerRun = zeichneKreiseFuerRun;
1071 window.zeichneFwertPunkte = zeichneFwertPunkte;
1072 window.zeichneDemoKreisgrafik = zeichneDemoKreisgrafik;
1073 window.merkeKreis = merkeKreis;
1074 window.vergissGezeichneteKreise = vergissGezeichneteKreise;
1075 window.demoIkonGetroffen = demoIkonGetroffen;
1076 window.zeichneKreisErklaerung = zeichneKreisErklaerung;
1077
1078 }()); // Ende der Modulkapselung, siehe Kommentar oben
```